

**Ruprecht-Karls-Universität Heidelberg**  
**Institut für Informatik**

**Bachelor Thesis**  
**Parallel ASP Planning**

Name: Levi Maximilian Klein  
Student ID: 4283957  
Supervisor: Dr. Gnad  
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

---

## **Zusammenfassung**

Dies ist eine Zusammenfassung der Arbeit.

## **Abstract**

This is the abstract.



# Contents

|                                 |          |
|---------------------------------|----------|
| <b>List of Figures</b>          | <b>1</b> |
| <b>List of Tables</b>           | <b>2</b> |
| <b>1 Einleitung</b>             | <b>3</b> |
| 1.1 Motivation . . . . .        | 3        |
| 1.2 Ziele der Arbeit . . . . .  | 3        |
| 1.3 Aufbau der Arbeit . . . . . | 3        |
| <b>2 Basics</b>                 | <b>4</b> |
| 2.1 Definitions . . . . .       | 4        |
| <b>Bibliography</b>             | <b>7</b> |



## List of Figures

## List of Tables

# 1 Einleitung

This chapter contains an overview of the topic as well as the goals and contributions of your work.

## 1.1 Motivation

## 1.2 Ziele der Arbeit

## 1.3 Aufbau der Arbeit

Here you describe the structure of the thesis. For example:

In Kapitel 2 werden grundlegende Methoden für diese Arbeit vorgestellt.

## 2 Basics

### 2.1 Definitions

**Definition 1 *Multi-valued planning task*.** A STRIPS-like multivalued planning task according to (Helmert 2006) is given by a 4 tuple  $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$ , where:

- $\mathcal{F}$  is a finite set of state variables, also called **fluents**. Each  $x \in \mathcal{F}$  has a finite domain  $x^d$  and in every state,  $x$  takes exactly one value from its domain.
- $s_0$ , the **initial state**, is a total function, s.t.  $s_0(x) \in x^d$  for each  $x \in \mathcal{F}$
- $s_*$ , the **goal conditions**, is a partial function, s.t.  $s_*(x) \in x^d$ , for each  $x \in \text{dom}(s_*)$
- $\mathcal{O}$  is a finite set of operators, also called **actions**. We define an action as  $a = \langle a^c, a^e \rangle$ , where  $a^c$  is the precondition of  $a$  and  $a^e$  is the postcondition of  $a$ . Both  $a^c$  and  $a^e$  are partial variable assignments over  $\mathcal{F}$ .

**Example.** Consider the following example for a plan to leave the house:

- $\mathcal{F} = \{\text{door}, \text{location}\}$  with  $\text{door}^d = \{\text{open}, \text{closed}\}$ ,  $\text{location}^d = \{\text{inside}, \text{outside}\}$
- Initial state:  $s_0 = \{\text{door}=\text{closed}, \text{location}=\text{inside}\}$
- Goal:  $s_* = \{\text{door}=\text{closed}, \text{location}=\text{outside}\}$
- Actions:  $\mathcal{O} = \{\text{openDoor}, \text{closeDoor}, \text{goOut}\}$ , where

$$\text{openDoor} = \langle \{\text{door}=\text{closed}\}, \{\text{door}=\text{open}\} \rangle$$

$$\text{closeDoor} = \langle \{\text{door}=\text{open}\}, \{\text{door}=\text{closed}\} \rangle$$

$$\text{goOut} = \langle \{\text{location}=\text{inside}, \text{door}=\text{open}\}, \{\text{location}=\text{outside}\} \rangle$$

**Definition 2 *Successor state*.** Let  $s$  be state and  $a \in \mathcal{O}$  an action.

The *successor state*  $o(a, s)$ , obtained by applying action  $a$  in state  $s$ , is defined if the precondition of  $a$  holds, i.e.,  $a^c(x) = s(x)$  for all  $x \in \text{dom}(a^c)$ . Then

$$s'(x) = o(a, s) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s(x) & \text{otherwise.} \end{cases}$$

*Remark.* Let  $A_n = \langle a_1, \dots, a_n \rangle$  be a sequence of actions with length  $n$ . We define the application of  $A_n$  to a state  $s$  recursively by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s))$$

if  $o(A_i, s)$  is defined for all  $1 \leq i \leq n$ .

**Definition 3 Sequential plan.** A *sequential plan* is a sequence of actions  $A_n = \langle a_1, \dots, a_n \rangle$ , s.t.  $s' = o(A_n, s_0)$  is defined and  $s'(x) = s_*(x)$  for all  $x \in \text{dom}(s_*)$ .

**Example.** Consider the planning task from the previous example. A sequential plan would be  $A = \langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle$ . Since

$$\begin{aligned} o(A, s_0) &= o(\langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{closed}, \text{location}=\text{inside}\}) \\ &= o(\langle \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{open}, \text{location}=\text{inside}\}) \\ &= o(\text{closeDoor}, \{\text{door}=\text{open}, \text{location}=\text{outside}\}) \\ &= \{\text{door}=\text{closed}, \text{location}=\text{outside}\} = s_* \end{aligned}$$

To get to parallel actions and plans, we need the definition of confluent actions.

**Definition 4 confluent.** Let  $A = \{a_1, \dots, a_n\} \subseteq \mathcal{O}$  be a set of actions.

$A$  is *confluent* : $\iff a_i^e(x) = a_j^e(x)$  for all  $1 \leq i < j \leq n$  and each  $x \in \text{dom}(a_i^e) \cap \text{dom}(a_j^e)$

*Remark.*  $A$  is confluent  $\iff B$  is confluent for all  $B \subseteq A$

**Example.** In our example  $\{\text{openDoor}, \text{goOut}\}$  and  $\{\text{closeDoor}, \text{goOut}\}$  are confluent sets. We extend our example with the fluents *light* and *key* with  $\text{light}^d = \text{key}^d = \{0, 1\}$  and two actions *lightsOut* and *getKey* with  $\text{lightsOut} = \langle \{\text{location}=\text{inside}, \text{light}=1\}, \{\text{light}=0\} \rangle$   
 $\text{getKey} = \langle \{\text{location}=\text{inside}, \text{key}=0\}, \{\text{key}=1\} \rangle$  Finally  $\text{openDoor}^c = \{\text{door}=\text{closed}, \text{key}=1\}$ . Our new goal is  $s_* = \{\text{location}=\text{outside}, \text{door}=\text{closed}, \text{light}=0\}$ . Now for example  $\{\text{getKey}, \text{openDoor}, \text{goOut}, \text{lightsOut}\}$  and  $\{\text{getKey}, \text{closeDoor}, \text{goOut}, \text{lightsOut}\}$  are also confluent sets.

**Example.** In our example  $\{\text{openDoor}, \text{goOut}\}$  and  $\{\text{closeDoor}, \text{goOut}\}$  are confluent sets. We extend our example with the fluents *light* and *key* with  $\text{light}^d = \text{key}^d = \{0, 1\}$  and two actions *lightsOut* and *getKey*:

$$\begin{aligned} \text{lightsOut} &= \langle \{\text{location}=\text{inside}, \text{light}=1\}, \{\text{light}=0\} \rangle \\ \text{getKey} &= \langle \{\text{location}=\text{inside}, \text{key}=0\}, \{\text{key}=1\} \rangle \end{aligned}$$

Additionally, we modify the precondition:  $\text{openDoor}^c = \{\text{door}=\text{closed}, \text{key}=1\}$ . Our new goal is  $s_* = \{\text{location}=\text{outside}, \text{door}=\text{closed}, \text{light}=0\}$ .

Now for example  $\{\text{getKey}, \text{openDoor}, \text{goOut}, \text{lightsOut}\}$  and  $\{\text{getKey}, \text{closeDoor}, \text{goOut}, \text{lightsOut}\}$  are also confluent sets.

**Definition 5 Parallel representations.** Let  $s$  be a state and  $A = \{a_1, \dots, a_n\}$  be a confluent set of actions. Then  $A$  is

- (i)  $\forall$ -step serializable in  $s$  : $\iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$  is defined for all permutations  $\pi$

- (ii)  $\exists$ -step serializable in  $s : \iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$  is defined for some permutation  $\pi[t]$  and  $a^c(x) = s(x)$  for each  $a \in A$  and  $x \in \text{dom}(a^c)$
- (iii) relaxed  $\exists$ -step serializable in  $s : \iff o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$  is defined for some permutation  $\pi$

*Remark.*  $A$  is  $\forall$ -step serializable  $\Rightarrow A$  is  $\exists$ -step serializable  $\Rightarrow A$  is relaxed  $\exists$ -step serializable.

**Definition 6 Parallel Plans.** Let  $A = \langle A_1, \dots, A_n \rangle$  be a sequence of confluent sets of actions.  $A$  is a  $\forall$ -step,  $\exists$ -step or relaxed  $\exists$ -step plan iff

- $s_n(x) = s_*(x)$  for each  $x \in \text{dom}(s_*)$
- $A_i$  is  $\forall$ -step,  $\exists$ -step or relaxed  $\exists$ -step serializable in  $s_{i-1}$  for  $1 \leq i \leq n$ , where
$$s_i(x) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s_{i-1}(x) & \text{else} \end{cases} \quad \text{for each } a \in A_i$$

**Example.** In our modified example we have:

- $\forall$ -step plan:  $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut\}, \{closeDoor\} \rangle$
- $\exists$ -step plan:  $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut, closeDoor\} \rangle$
- relaxed  $\exists$ -step plan:  $\langle \{getKey, lightsOut, openDoor, goOut\}, \{closeDoor\} \rangle$

# Bibliography

Helmert, M. (2006). “The Fast Downward Planning System”. In: *Journal of Artificial Intelligence Research* 26, pp. 191–246.